

ESERCIZIO 1: Consideriamo la rappresentazione dei numeri interi con segno "in complemento a 2" (abbreviato con C2)

- A). Da cosa capisco se un numero rappresentato su K bit in C2 è positivo o negativo?
- B). Se ho un numero n in complemento a 2 e voglio trovare il suo opposto, (cioè $-n$, cioè il numero cambiato di segno) che tipo di operatore devo applicare al numero binario n di partenza per ottenere la rappresentazione in complemento a 2 del suo opposto?
- C). Consideriamo i due seguenti numeri su 6 bit, rappresentati in complemento a 2:
 $n1 = 111111$ $n2 = 010100$
 - i). I segni dei due numeri sono concordi o discordi?
 - ii). calcolare la somma dei due numeri (espressi sempre in C2 su 6 bit) e discutere se si verifica overflow.
- D). Se ho un registro di 16 bit e devo memorizzare un numero di 6 bit al suo interno, come faccio ad estenderlo in modo da riempire tutti i bit del registro? Fare l'esempio con i due numeri indicati sopra

A) IL COMPLEMENTO A 2 CODIFICA I NUMERI BINARI CON SEGNO, UTILIZZANDO IL BIT PIÙ SIGNIFICATIVO PER IL SEGNO (SE È 0 = +, 1 = -) E I RESTANTI $K-1$ BIT CON IL MODULO DEL NUMERO. QUINDI BASTA CONTROLLARE IL BIT PIÙ SIGNIFICATIVO (QUELLO + A SINISTRA).

B) SE HO n IN C2 E VOGLIO TROVARE IL SUO OPPOSTO, BASTA APPLICARE UN NOT AD OGNI CIFRA DI n (FARE IL COMPLEMENTO A 1) E AGGIUNGERE +1 AL NUMERO (DI SOLITO QUESTO +1 È IL CARRY INIZIALE NELLA CPU QUANDO FA OPERAZIONI DI SOMMA).

C) $N1 = 111111$ $N2 = 010100$ $N \text{ BIT} = 6$

i) I DUE NUMERI SONO DISCORDI PERCHÉ HANNO IL BIT PIÙ SIGNIFICATIVO DIVERSO.

ii)

$$\begin{array}{r} \cancel{1}11 \\ 111111 \\ 010100 \\ \hline 010011 \end{array} +$$

NON SI VERIFICA OVERFLOW PERCHÉ L'ULTIMO CARRY È UGUALE AL PENULTIMO

($C_{IN} = C_{OUT}$ NEL BIT DEL SEGNO)

D) SE HO UN REGISTRO A 16 BIT E VOGLIO MEMORIZZARE UN NUMERO IN C2 BISOGNA ESTENDERE IL BIT DEL SEGNO SUI RESTANTI BIT A SINISTRA, QUINDI CON UN POSITIVO TOT ZERI E CON UN NEGATIVO TOT 1.

CON $N1$ DOVREI METTERE IL NUMERO NELLE ULTIME 6 POSIZIONI E RIEMPIRE A SINISTRA CON IL SEGNO 1, ALLO STESSO MODO CON $N2$ HA RIEMPIENDO A SINISTRA CON IL SEGNO 0.

$N1$ IN 16 BIT: 1111 1111 1111 1111

$N2$ IN 16 BIT: 0000 0000 0001 0100

ESERCIZIO 2:

- 1) Si consideri il seguente numero frazionario in base 10: 4,3125
 - trovare la rappresentazione in base 2 di tale numero e poi trasformare questa ultima in base 16.
- 2) Rappresentare il numero binario seguente -110,11011101 in floating point ieee754 precisione singola (in binario, su 32 bit). Spiegare tutti i passaggi

$$1) N_1 = 4,3125 \quad 4 = 100 \quad \begin{array}{l} 0,3125 \cdot 2 = 0,625 \\ 0,625 \cdot 2 = 1,25 \\ 0,25 \cdot 2 = 0,5 \\ 0,5 \cdot 2 = 1,0 \end{array}$$

$$N_{1(2)} = 100,0101$$

VISTO CHE SE RAGGRUPPIAMO 4 BIT IN BINARIO FORMIAMO UN NUMERO ESADECIMALE: $N_{1(2)} = \underset{4}{0100} \mid \underset{5}{0101} = 4,5_{16}$

$$2) N_2 = -110,11011101$$

1. IL BIT DEL SEGNO È 1 (PERCHÉ NEGATIVO)
2. $|N| = 110,11011101$
3. **NORMALIZZO:** $1,1011011101 \cdot 2^2$ (SPOSTATO A SINISTRA DI 2 POSIZIONI)
4. TROVO L'ESPONENTE IN ECCESSO 127: $R = 2 + 127 = 129$
 $129_{10} = 10000001$
5. **RISULTATO:** $1 \mid 10000001 \mid 101101110100000000000000$

DOMANDA 1:

- 1) Spiegare cosa si intende con l'espressione "ciclo di fetch decode execute" (in italiano prelievo, decodifica esecuzione) e indicare chi, all'interno di un computer, esegue continuamente questo ciclo.
- 2) A cosa serve il clock in un computer? Qual è la caratteristica del clock che influenza la velocità di esecuzione dei programmi?

- 1) IL CICLO DI FETCH DECODE EXECUTE INDICA QUEL PROCEDIMENTO SVOLTO DALLA CPU (E DA CU E ALU) DI PRELIEVO DI ISTRUZIONI, EVENTUALI DATI, DECODIFICA DELL'ISTRUZIONE ED ESECUZIONE, CHE SI DIVIDE IN DIVERSI PASSI:
 - 1) **FETCH:** L'ISTRUZIONE VIENE PRELEVATA DAL PC (PROGRAM COUNTER) E INSERITA NEL IR (INSTRUCTION REGISTER).
 - 2) IL PC VIENE INCREMENTATO PER "PUNTARE" ALLA PROSSIMA ISTRUZIONE.
 - 3) **DECODE:** LA CU DECODIFICA L'ISTRUZIONE PRELEVATA E SE SERVONO ULTERIORI DATI VENGONO PRESI IN MEMORIA (SE NON GIÀ PRESENTI NEI REGISTRI)
 - 4) **EXECUTE:** L'ALU ESEGUE L'ISTRUZIONE E RESTITUISCE UN RISULTATO
 - 5) SI TORNA AL PASSO 1 (FETCH)

2) IL CLOCK È UN COMPONENTE MOLTO IMPORTANTE E SERVE PER ORDINARE EVENTI CHE HANNO BISOGNO DI ESSERE ESEGUITI IN MODO SEQUENZIALE.

LA CARATTERISTICA DEL CLOCK CHE INFLUENZA LA VELOCITÀ DEI PROGRAMMI È LA SUA FREQUENZA INFATTI LA FORMULA PER CALCOLARE IL TEMPO DI CPU DEDICATO AL PROGRAMMA SI CALCOLA COME:

$$T_{CPU} = \frac{I \cdot CPI}{f}$$

I → N° ISTRUZIONI DEL PROGRAMMA
 CPI → N° DI CICLI MEDIO PER OGNI ISTRUZIONE
 f → FREQUENZA DI CLOCK
 $I \cdot CPI$ → N° DI CICLI DI CLOCK PER ESEGUIRE IL PROGRAMMA

DOMANDA 2:

Quali sono le differenze tra le seguenti rappresentazioni di caratteri: (a) ASCII base, (b) ASCII esteso (nelle sue diverse versioni) e quelle basate sui codepoint Unicode: (c) UCS-2 e (d) UTF-8?

Perché è stato introdotto lo standard UNICODE nonostante esistessero già i codici ASCII esteso per molte lingue?

A) L'ASCII BASE (AMERICAN STANDARD FOR INFORMATION INTERCHANGE) È LA PRIMA RAPPRESENTAZIONE DEI CARATTERI, SI BASA SULL'UTILIZZO DI 7 BIT, QUINDI INTUTTO PUÒ RAPPRESENTARE 2^7 CARATTERI (128 CARATTERI).

B) L'ASCII ESTESO È UNA VERSIONE ESTESA DELL'ASCII BASE, UTILIZZA 8 BIT, QUINDI PUÒ RAPPRESENTARE FINO A 256 CARATTERI, NÈ ESISTONO DI DIVERSE VERSIONI CHIAMATE "CODE PAGES" (COME IL LATIN-1) E NON SI POSSONO INSERIRE CARATTERI DI DIVERSI CODEPAGES NELLO STESSO TESTO.

C) L'UNICODE VENNE INTRODOTTO PER STANDARDIZZARE LA CODIFICA DI CARATTERI DI TESTO, UTILIZZA I CODEPOINT, OVERO CODICI UNIVOCI PER OGNI CARATTERE E SIMBOLO. LA PRIMA VERSIONE A 16 BIT NON BASTÒ (65536 CARATTERI) QUINDI VENNERO AGGIUNTI ALTRI 16 PIANI DIVERSI PER RAGGIUNGERE PIÙ DI UN MILIONE DI CODEPOINT ($2^{16} \times 17$).

LA CODIFICA IN UCS-2 ERA LA PRIMA VERSIONE DI CUI ABBIAMO PARLATO, SU 2 BYTE A LUNGHEZZA FISSA (LIMITATO UNICODE 1.0 SU 16 BIT)

D) L'UTF-8 È UNA CODIFICA UNICODE A LUNGHEZZA VARIABILE SU 1 A 6 BYTE, E TUTTI I BYTE SUCCESSIVO AL PRIMO INIZIANO CON 10. LO SVANTAGGIO È CHE ESSENDO A LUNGHEZZA VARIABILE È PIÙ DIFFICILE SAPERE DOVE INIZIA E DOVE FINISCE, MA IN CAMBIO HA:

- PREFISSI DIVERSI TRA IL PRIMO E I SUCCESSIVI BYTE (NON SI PERDE L'ALLINEAMENTO)
- SI RISPARMIA SPAZIO RISPETTO ALLE CODIFICHE A LUNGHEZZA FISSA
- COMPATIBILE CON ASCII
- NON È LEGATO ALL'ORDINE DEI BYTE.